

COS30018 - Option C - Task 5 Report

Machine Learning 1

Student Name: Kha Anh Nguyen

Student ID: 103814796

Date: September 5, 2025

Course: COS30018 - Intelligent Systems

Project: FinTech101 Stock Price Prediction System

Github repo: [Leissha/stock_prediction](https://github.com/Leissha/stock_prediction)

Table of Contents

COS30018 - Option C - Task 5 Report	1
1. Key changes:.....	1
2. Codebase restructured	1
3. TFModel update	5
4. SARIMA	6
5. Ensemble Model	9
6. Hyperparameter tuning	12
7. Conclusion	21

1. Key changes:

- Restructured the whole codebase to a modular, single-source-of-truth architecture with validated data contracts.
- Add SARIMA and ensemble model
- Add train/val/test split & apply early stopping

2. Codebase restructured

Pydantic Data Schema

Before, the system passively add transformation code (inverse transform, reshape) depends on the model/functions requirements, which are very inefficient and unorganized.

Therefore, I restructured the entire codebase and created Pydantic DataBundle to control and audit the data shapes at each step from data preprocessing to model evaluation.

This help

- **Shape validation:** Active pre-enforce to ensure data consistency throughout the pipeline and catches shape mismatches early.
- **Single source of truth:** All transformations are processed in dataio/ folder

Below is the snippet of schema/bundle.py & validation:

```
class TargetMode(str, Enum):
    PRICE = "price"
    RETURN = "return"
    LOG_RETURN = "log_return"

class DataBundle(BaseModel):
    model_config = ConfigDict(arbitrary_types_allowed=True)

    # Required train/test data
    X_train: NDArray # Shape: (N, L, F)
    y_train: NDArray # Shape: (N, K)
    X_test: NDArray # Shape: (N, L, F)
    y_test: NDArray # Shape: (N, K)

    # Optional validation data
    X_val: Optional[NDArray] = None
    y_val: Optional[NDArray] = None
    base_prices_val: Optional[NDArray] = None

    # Metadata
    symbol: str
    features: List[str]
    lookback: int = Field(gt=0)
    horizon: int = Field(gt=0)
    target_mode: TargetMode
    use_log_returns: bool = False
    scalers: Dict[str, Any] = Field(default_factory=dict)
    target_scaler: Optional[Any] = None
    base_prices_test: Optional[NDArray] = None

    # Optional fields for SARIMAX (original DataFrames before sequencing)
    train_df: Optional[Any] = None # pd.DataFrame
    test_df: Optional[Any] = None # pd.DataFrame
    val_df: Optional[Any] = None # pd.DataFrame
```

```

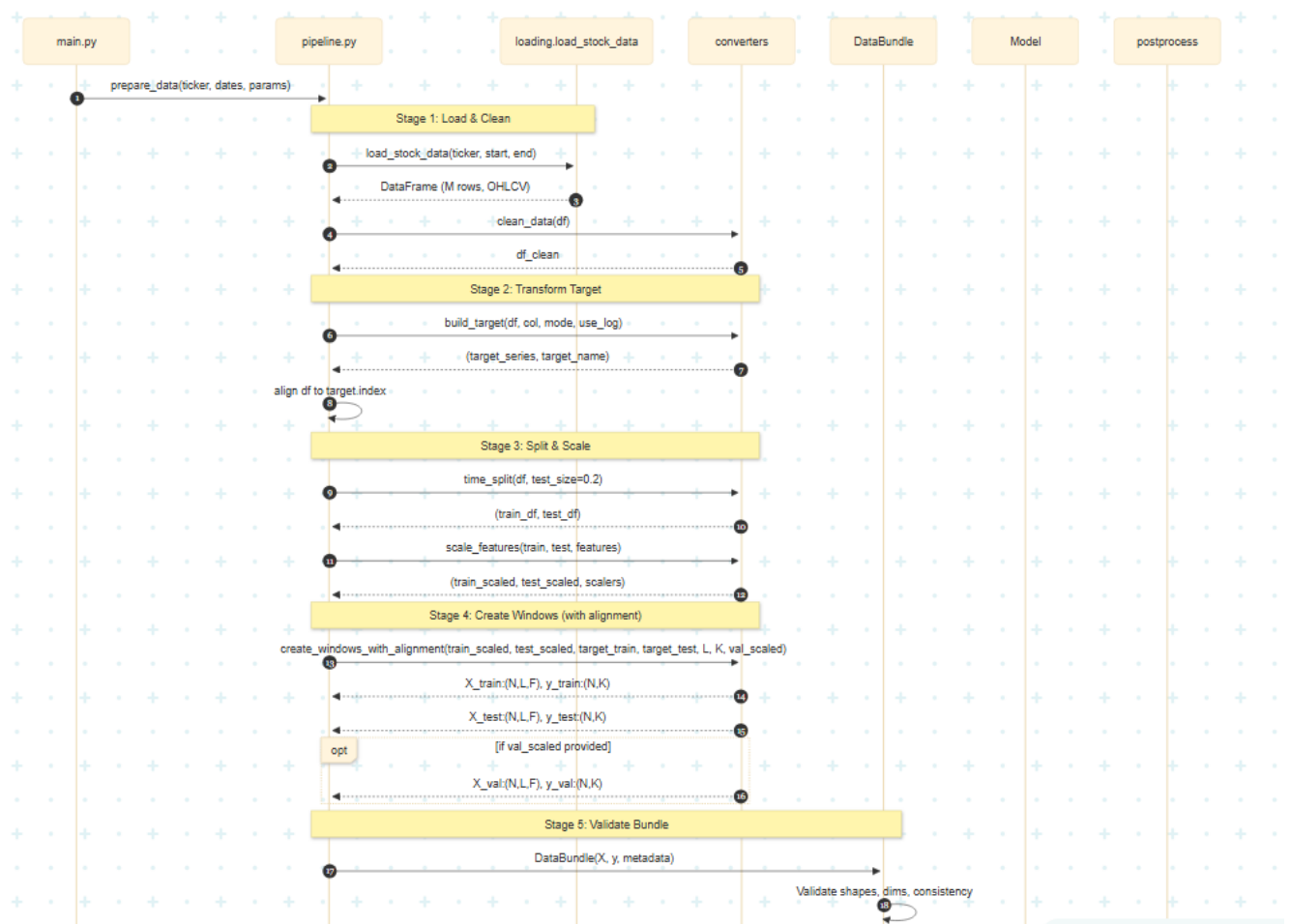
target_feature: Optional[str] = None

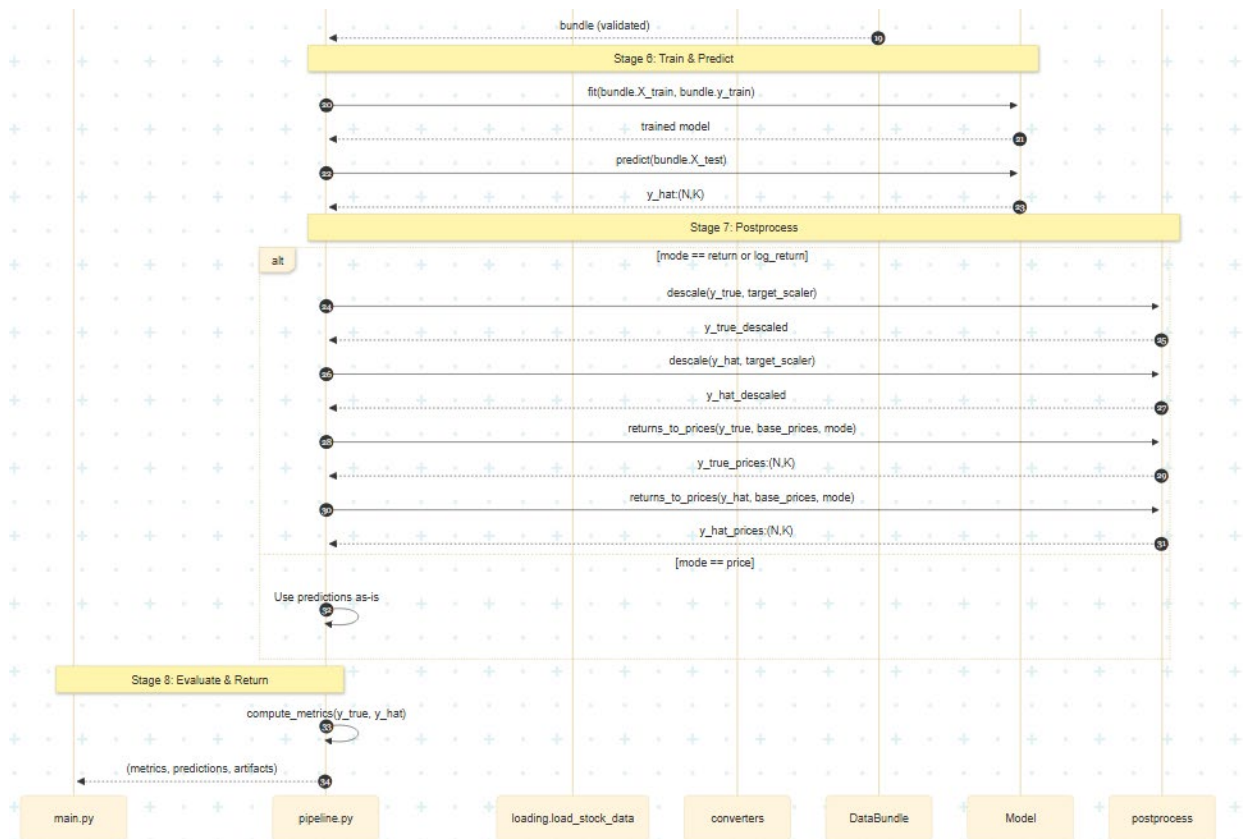
@model_validator(mode='after')
def validate_all(self):
    # ---- Ranks ----
    if self.X_train.ndim != 3:
        raise ValueError(f"X_train must be 3D (N,L,F), got shape {self.X_train.shape}")
    if self.X_test.ndim != 3:
        raise ValueError(f"X_test must be 3D (N,L,F), got shape {self.X_test.shape}")
    and more validations...

```

Single source of truth:

Below shows the current modular architecture data sequence, no more transformation functions are scattered across multiple files (data_preprocessing, train.py, test.py, utils...):





Module's responsibilities:

Module	Responsibility	Key Functions
dataio/converters.py	Data preprocessing pipeline	clean_data(), build_target(), time_split(), scale_features(), windows_train_test(), windows_train_val_test()
dataio/postprocess.py	Post-processing transformations	descale(), returns_to_prices(), align_predictions()
dataio/loading.py	Data loading and caching	load_stock_data()
schemas/bundle.py	Data contract validation	DataBundle (Pydantic)
pipeline.py	Orchestration layer	prepare_data(), predict(), predictions_to_prices()
eval/metrics.py	Evaluation metrics	compute_metrics()
model/sarimax.py	SARIMA implementation	SARIMAXModel
model/ensemble.py	Ensemble model	EnsembleModel

Dataio

Most of the code are refactor from the old code, so I am not document the code here but only explain the refactor logic.

Converter:

Refactor from the old dataPreprocessor class, the converter handle all data transformations and validates data shape with DataBundle.

- **clean_data():** Handles missing OHCLV values
- **build_target():** Creates price/return/log_return targets with proper formulas
- **time_split():** Creates chronological train/test/val splits
- **scale_features():** Fits scalers on training data only (prevents data leakage)
- **windows_train_test():** Creates sliding window sequences $(N,L,F) \rightarrow (N,K)$
- **windows_train_val_test():** Single entrypoint to produce (X_{train}, y_{train}) , (X_{val}, y_{val}) , (X_{test}, y_{test}) .

Postprocessor:

- **descale():** Inverse StandardScaler transformation (if used)
- **returns_to_prices():** Convert returns/log_returns back to actual prices using per-row base prices
- **align_predictions():** Aligns predictions with ground truth

Pipeline

- **prepare_data():** Orchestrates dataio steps and returns a validated DataBundle
- **predict():** Calls model.predict on bundle.X_test $\rightarrow (N,K)$
- **predictions_to_prices():** Converts y_{true}/y_{hat} to price space (uses descale + returns_to_prices with bundle.base_prices_test)

3. TFModel update

TFModel with EarlyStopping and train/val:

```
def fit(self, x_train, y_train, epochs=25, batch_size=32, learning_rate=1e-3, optimizer:
str = 'adam', validation_data=None, meta_path=None):
    """Train a TensorFlow model using built-in Keras features.
    validation_data: optional tuple (X_val, y_val) to monitor validation loss.
    """
    # Compile model
    optimizers = {'adam': Adam, 'rmsprop': RMSprop, 'sgd': SGD}
```

```

opt_cls = optimizers[optimizer.lower()]
opt = opt_cls(learning_rate=learning_rate)
self.model.compile(optimizer=opt, loss='mean_squared_error', metrics=['mae'])

# Train and capture history
fit_kwargs = {
    'x': x_train,
    'y': y_train,
    'epochs': epochs,
    'batch_size': batch_size,
    'verbose': 1,
}

if validation_data is not None:
    fit_kwargs['validation_data'] = validation_data
# Early stopping if validation is provided
callbacks = []
if validation_data is not None:
    callbacks.append(EarlyStopping(monitor='val_loss', patience=20,
restore_best_weights=True, mode='min'))

history = self.model.fit(**fit_kwargs, callbacks=callbacks)

plot_training_metrics(history,
save_path=f"cache/trained_models/{meta_path}_training.png")
return self

```

4. SARIMA

I chose to use ARIMA/SARIMA instead of SARIMAX as they need exogenous value for the prediction, while we only have (OHCLV) features, which would lead to data leakage. Hence, the current SARIMA is mainly for linear relationship correlation and single input of the user choice (OHCLV), default is Close.

Understand how SARIMA works: ACF, PACF & ADF statistics

ACF (Autocorrelation function): Helps identify MA components (q)

PACF (Partial Autocorrelation function): Helps identify AR components (p)

ADF (Augmented Dickey-Fuller) Test: Helps determine differencing order (d)

ACF plot and PACF plot

- **ACF** (autocorrelation function) is the correlation of the time series with its lags, e.g., y_t and y_{t-k} for $k = 1, 2, \dots$

Question: assume y_t and y_{t-1} are correlated, y_{t-1} and y_{t-2} are correlated.

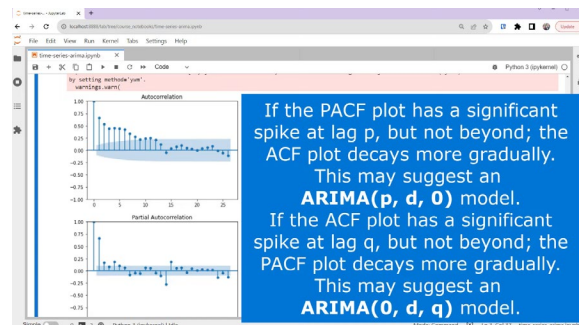
How can we measure if there is new information in y_{t-2} to predict y_t , besides their relationships with y_{t-1} ?

- **PACF** (partial autocorrelation function) is the partial correlation of the time series with its lags, after removing the effects of lower-order-lags between them

e.g., the partial autocorrelation of y_t and y_{t-k} is the correlation that is not explained by their relationships with the lags $y_{t-1}, y_{t-2}, \dots, y_{t-k+1}$

Figure 1-4 Youtube theory & code Source:

https://youtu.be/aCF0_wfVwY?si=7KEE08wy7NNARoxk

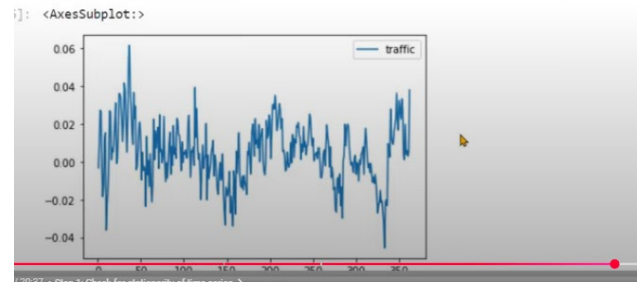


Method #3: ADF test

```
from statsmodels.tsa.stattools import adfuller
adf_test = adfuller(df_train)
print(f'p-value: {adf_test[1]}')
p-value: 0.24126116082883903
```

Transform to stationary: differencing

```
df_train_diff = df_train.diff().dropna()
df_train_diff.plot()
```



Optional: Auto-fit the ARIMA model

```
import pmdarima as pm
auto_arima = pm.auto_arima(df_train, stepwise=False, seasonal=False)
auto_arima
```

auto_ARIMA

- Instead of doing grid search manually to find the p, d, q level based on the above statistics, I use statsforecast.Auto_ARIMA to help me handle the search and select the (p, d, q) and (P, D, Q, m) parameters automatically.

SARIMA

- The parameters will be added to statsmodel.SARIMA for fitting with unscaled data.

Auto_ARIMA + SARIMA combo:

```
ARIMA/SARIMA Model Implementation for Time Series Forecasting
References:
- how it works: https://youtu.be/dr1t0pNEUH4?si=rsm6GqGsxmFVhXE5
- log, pacf plots explanation:
https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html
- library: https://www.statsmodels.org/stable/generated/statsmodels.tsa.statespace.sarimax.SARIMAX.html
- code guide: https://medium.com/@injure21/arima-for-anomaly-detection-85bdf5d585
```

- <https://github.com/liannewriting/YouTube-videos-public/blob/main/arma-model-time-series-prediction-python/time-series-arma.ipynb>
- <https://blog.quantinsti.com/forecasting-stock-returns-using-arma-model/>
- <https://www.kaggle.com/code/nageshsingh/stock-market-forecasting-arma>
- <https://www.machinelearningplus.com/time-series/arma-model-time-series-forecasting-python/>

ARIMAX/SARIMAX implementation compatible with our training pipeline.

Notes:

- Uses unscaled target series from the DataBundle (endog only; no exogenous features to avoid leakage).
 - Input windows are 3D (N, lookback, features) but SARIMA only uses N to produce (N, K) forecasts.
 - Forecast horizon is aligned via `forecast_horizon=bundle.horizon` to match TF models.
 - Multi-step predictions are generated via rolling (walk-forward) forecasting with state updates.
- """

class SARIMAXModel:

```
def __init__(self, seasonal: bool = False, m: int = 5, forecast_horizon: int = 1):
    self.seasonal = bool(seasonal)
    self.m = int(m)
    self.forecast_horizon = int(forecast_horizon)
    self.order: Optional[Tuple[int, int, int]] = None # p, d, q
    self.seasonal_order: Optional[Tuple[int, int, int, int]] = None # P, D, Q, m
```

```
def fit(self, data=None):
```

```
    """Fit SARIMAX using the unscaled target series from the bundle."""
```

```
    # Extract target series from DataBundle
```

```
    target_series = data['train_df'][data['target_feature']].astype(float).values
```

```
    # Auto-select parameters using statsforecast.AutoARIMA
```

```
    auto_arima = AutoARIMA(
        season_length=self.m if self.seasonal else 1,
        max_p=3, max_q=3, max_P=2, max_Q=2, max_d=2,
        stepwise=True, approximation=False, seasonal=self.seasonal
    )
```

```
    auto_arima.fit(target_series)
```

```
    # Extract selected parameters
```

```
    arma = auto_arima.model_['arma']
```

```
    self.order = (arma[0], arma[1], arma[2]) # p, d, q
```

```
    self.seasonal_order = (arma[3], arma[4], arma[5], arma[6]) # P, D, Q, m
```

```
    # Fit statsmodels.SARIMAX with selected parameters
```

```
    model = SARIMAX(target_series, order=self.order, seasonal_order=self.seasonal_order)
```

```
    self._results = model.fit(dispatch=False)
```


Rolling prediction:

To allow multistep forecasting, the next day forecast would be based on the previous day.

```
def predict(self, x_test: np.ndarray) -> np.ndarray:
    """Generate rolling forecasts."""
    N, K = x_test.shape[0], self.forecast_horizon
    preds = np.zeros((N, K))

    for i in range(N):
        # Get forecast for K steps
        fc = self._results.get_forecast(steps=K)
        m = np.asarray(fc.predicted_mean)
        preds[i, :] = m[:K]

        # Walk-forward update with own 1-step forecast
        self._results = self._results.append(endog=np.array([m[0]]))

    return preds
```

5. Ensemble Model

To start simple, the ensemble model implements weighted average approach to combine a statistical model (SARIMA) and a deep learning model (TF: LSTM/GRU/RNN/BiLSTM) to capture complementary patterns: SARIMA for linear/seasonal structure; TF for non-linear temporal dependencies.

Weight ensemble

- **Interface:** All models implement `fit(X_train, y_train)` and `predict(X_test) → (N, K)`.
- **Data contract:** Predictions must match `bundle.horizon K`. We explicitly set `SARIMA forecast_horizon=bundle.horizon` to align shapes.
- **No extra transformation:** Models receive already windowed, scaled inputs from the pipeline and post-processing is centralized.

```
class EnsembleModel:
    """
    Ensemble that combines SARIMA and LSTM predictions via weighted average.
    Models must be pre-fitted and expose predict() that returns (N, K).
    """

    def __init__(self, config: EnsembleConfig):
```

```

self.config = config
self.sarima_model = None
self.model_2 = None
self.is_fitted = False

# Validate and normalize weights
if self.config.normalize_weights:
    total_weight = self.config.sarima_weight + self.config.model_2_weight
    if total_weight > 0:
        self.config.sarima_weight /= total_weight
        self.config.model_2_weight /= total_weight
    else:
        raise ValueError("At least one weight must be positive")

def fit(self, sarima_model, model_2):
    """
    Register pre-trained models.

    Args:
        sarima_model: Pre-trained SARIMA model with predict(X_test) -> (N,K)
        model_2: Pre-trained TF model with predict(X_test) -> (N,K)
    """
    self.sarima_model = sarima_model
    self.model_2 = model_2
    self.is_fitted = True

def predict(self, x_test):
    """Return weighted-average predictions with shape (N, K)."""
    if not self.is_fitted:
        raise RuntimeError("Ensemble model not fitted. Call fit() first.")

    if self.sarima_model is None or self.model_2 is None:
        raise ValueError("Both SARIMA and Model 2 models must be set")

    sarima_preds = self.sarima_model.predict(x_test)
    model_2_preds = self.model_2.predict(x_test)

    # Shape validation
    if isinstance(sarima_preds, tuple) or isinstance(model_2_preds, tuple):
        raise TypeError("Submodels must return numpy arrays, not tuples")
    if sarima_preds.ndim != 2 or model_2_preds.ndim != 2:
        raise ValueError( #error message )
    if sarima_preds.shape != model_2_preds.shape:
        raise ValueError( #error message )

```

```

# Weighted average
ensemble_preds = (
    self.config.sarima_weight * sarima_preds +
    self.config.model_2_weight * model_2_preds
)
return ensemble_preds

```

Build submodel from bundle (load SARIMAX and TFModel instances)

```

def fit_from_bundle(self, bundle, sarima_args: dict, model_2_args: dict,
model_2_meta_path: str):
    """
    Assemble and prepare submodels from the DataBundle.

    Args:
        bundle: DataBundle with train/val splits and metadata
        sarima_args: { 'seasonal': bool, 'm': int }
        model_2_args: { 'layers': List[int], 'dropout_rate': float, 'epochs': int,
'batch_size': int, 'model_name': str='lstm' }
        model_2_meta_path: filesystem path for saving/loading the TF model
    """
    # Import locally to avoid circular imports at module load time
    from model.sarimax import SARIMAXModel
    from model.tf_models import TFModel
    from tensorflow import keras as tf_keras # type: ignore

    # 1) SARIMA from bundle (unscaled target)
    sarima = SARIMAXModel(
        seasonal=bool(sarima_args.get('seasonal', False)),
        m=int(sarima_args.get('m', 5)),
        forecast_horizon=bundle.horizon,
    )
    data_dict = {
        'train_df': bundle.train_df,
        'target_feature': bundle.target_feature,
        'scalers': bundle.scalers,
        'test_df': bundle.test_df,
    }
    sarima.fit(data=data_dict)

    # 2) Model 2 from bundle (LSTM/GRU/RNN/BiLSTM)
    model_2 = TFModel(
        input_size=bundle.X_train.shape[2],

```

```

        model_name=str(model_2_args.get('model_name', 'lstm')),
        layers=model_2_args.get('layers'),
        dropout_rate=float(model_2_args.get('dropout_rate', 0.2)),
        output_steps=bundle.horizon,
    )
    model_2_model_path = f"cache/trained_models/{model_2_meta_path}.keras"
    if not os.path.exists(model_2_model_path):
        val_tuple = None
        if bundle.X_val is not None and bundle.y_val is not None:
            val_tuple = (bundle.X_val, bundle.y_val)

        fit_kwargs = {
            'epochs': int(model_2_args.get('epochs', 25)),
            'batch_size': int(model_2_args.get('batch_size', 32)),
            'validation_data': val_tuple,
            'optimizer': str(model_2_args.get('optimizer', 'adam')),
            'learning_rate': float(model_2_args.get('learning_rate', 1e-3)),
            'meta_path': model_2_meta_path,
        }

        model_2.fit(
            bundle.X_train,
            bundle.y_train,
            **fit_kwargs,
        )
        model_2.save_model(model_2_model_path)
    else:
        keras_model = tf.keras.models.load_model(model_2_model_path)
        model_2.model = keras_model

    # 3) Register
    self.fit(sarima, model_2)
    return self

```

6. Hyperparameter tuning

CLI examples

```

# SARIMA (univariate target)
python main.py --model_name sarimax --lookback 60 --horizon 5 --sarimax_seasonal --sarimax_m 5

# Ensemble: SARIMA + GRU (weighted average)
python main.py --model_name ensemble --ensemble_2 gru --lookback 60 --horizon 5 --layers 64 32
--dropout_rate 0.2 --epochs 50 --batch_size 32 --optimizer rmsprop --learning_rate 0.0005 --
sarima_weight 0.5 --model_2_weight 0.5 --scale --val_size 0.1 --log_ret

```

Test result example plots

1. RNN [50,50,50] horizon=1 (Best single-step model)

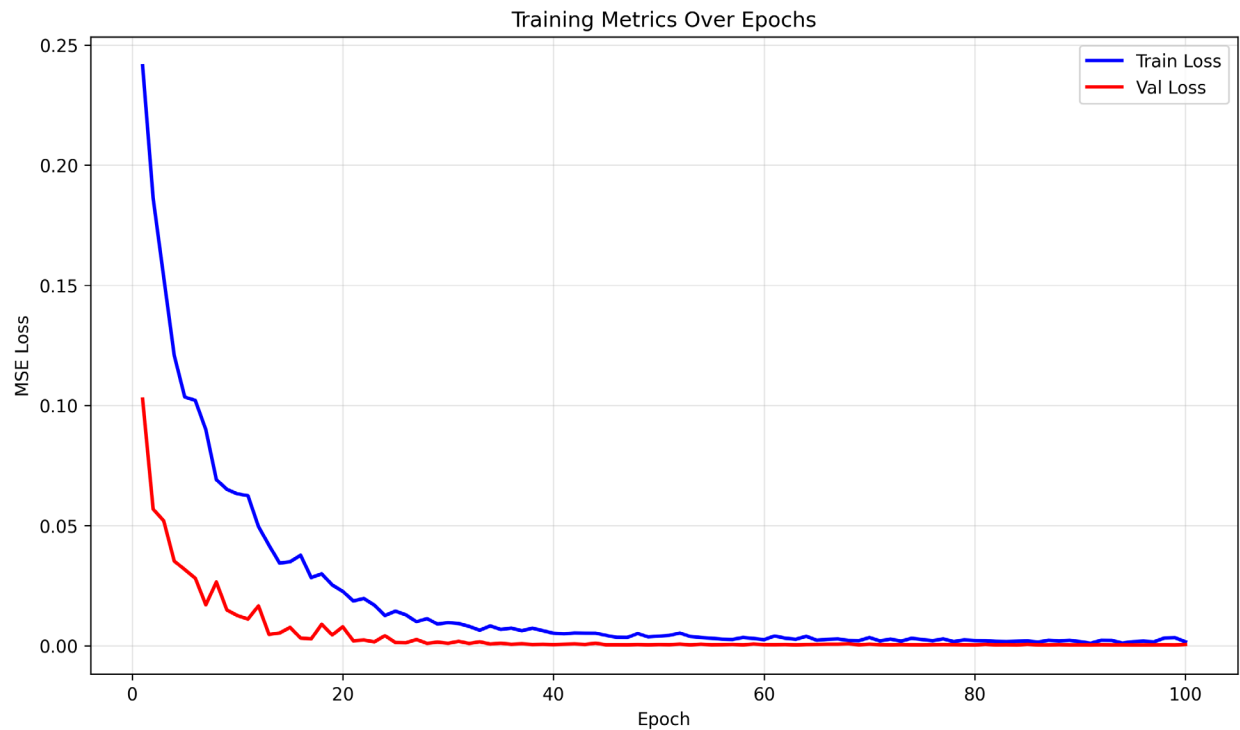


Figure 1. Best single-step MAE, smooth training, stable convergence (100 epochs)

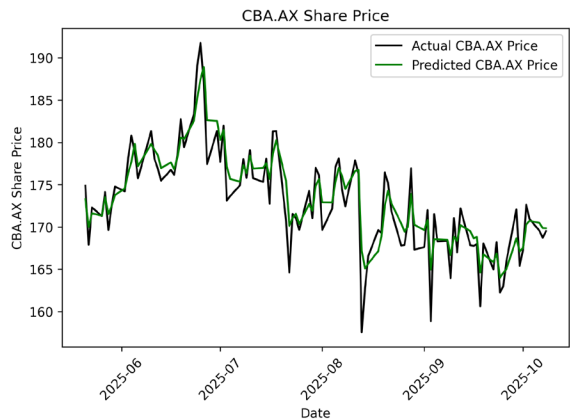


Figure 2. RNN [50,50,50] horizon=1 prediction

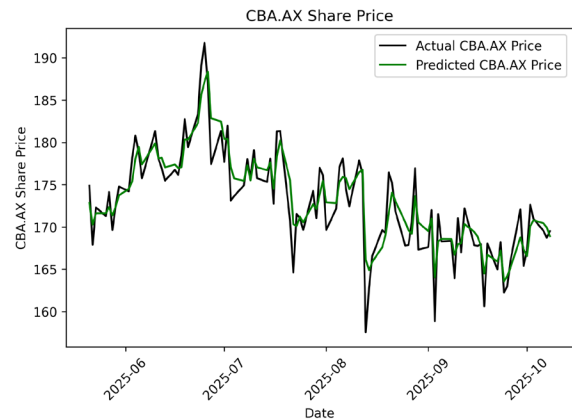


Figure 3. Ensemble (SARIMA:0.3 + RNN:0.7) [50,50,50] horizon=1 prediction

2. RNN [64,32] horizon=1

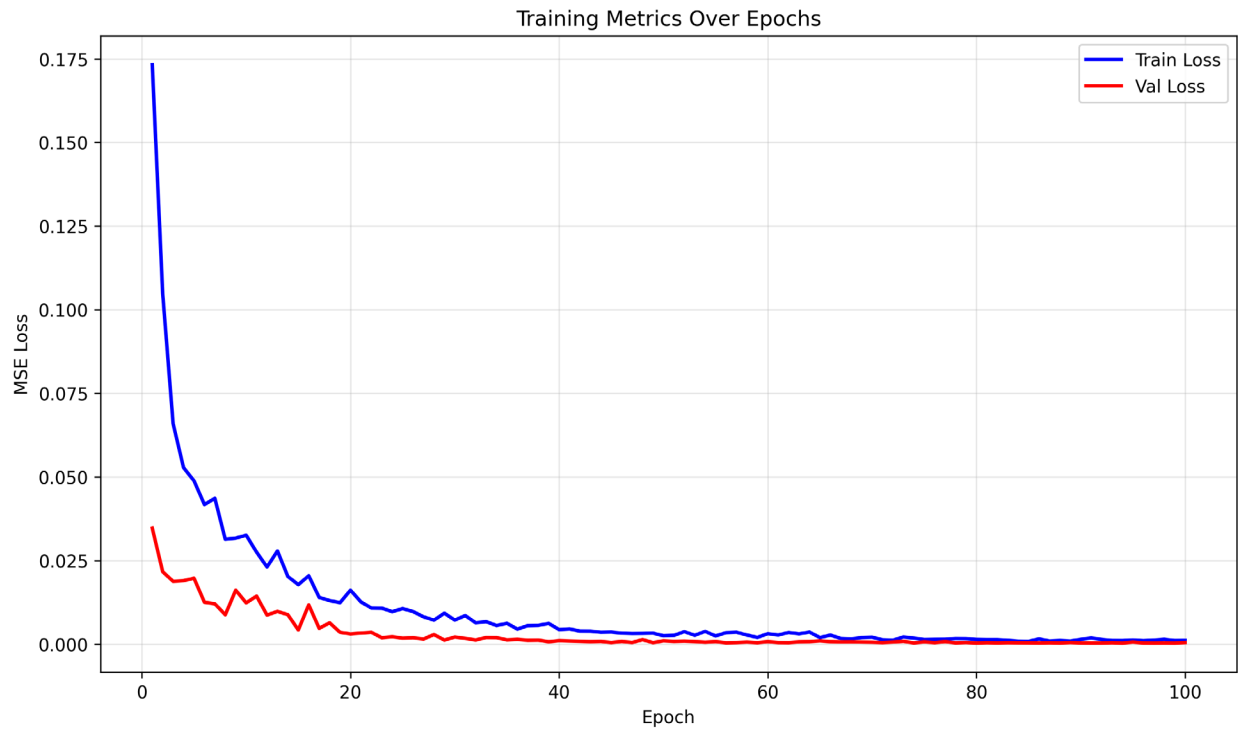


Figure 4. Training plot for RNN [64,32] horizon=1

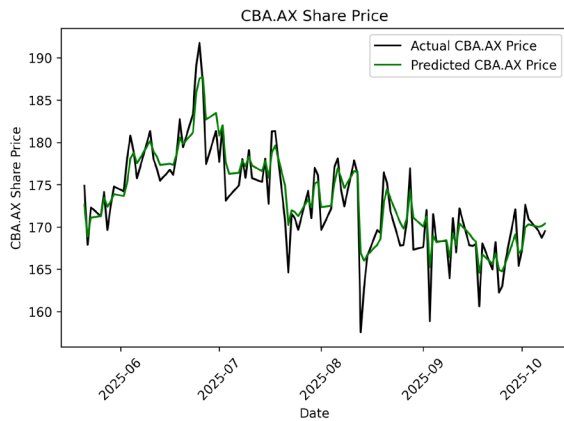


Figure 5. RNN [64,32] horizon=1 prediction

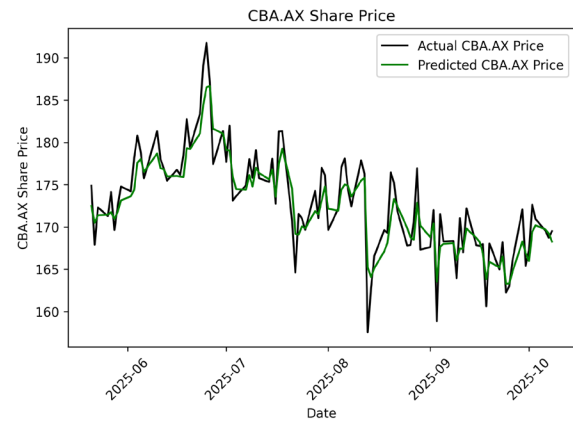


Figure 6. Ensemble (SARIMA:0.5 + RNN:0.5) [64,32] horizon=1 prediction

3. BiLSTM [50,50,50] horizon=1

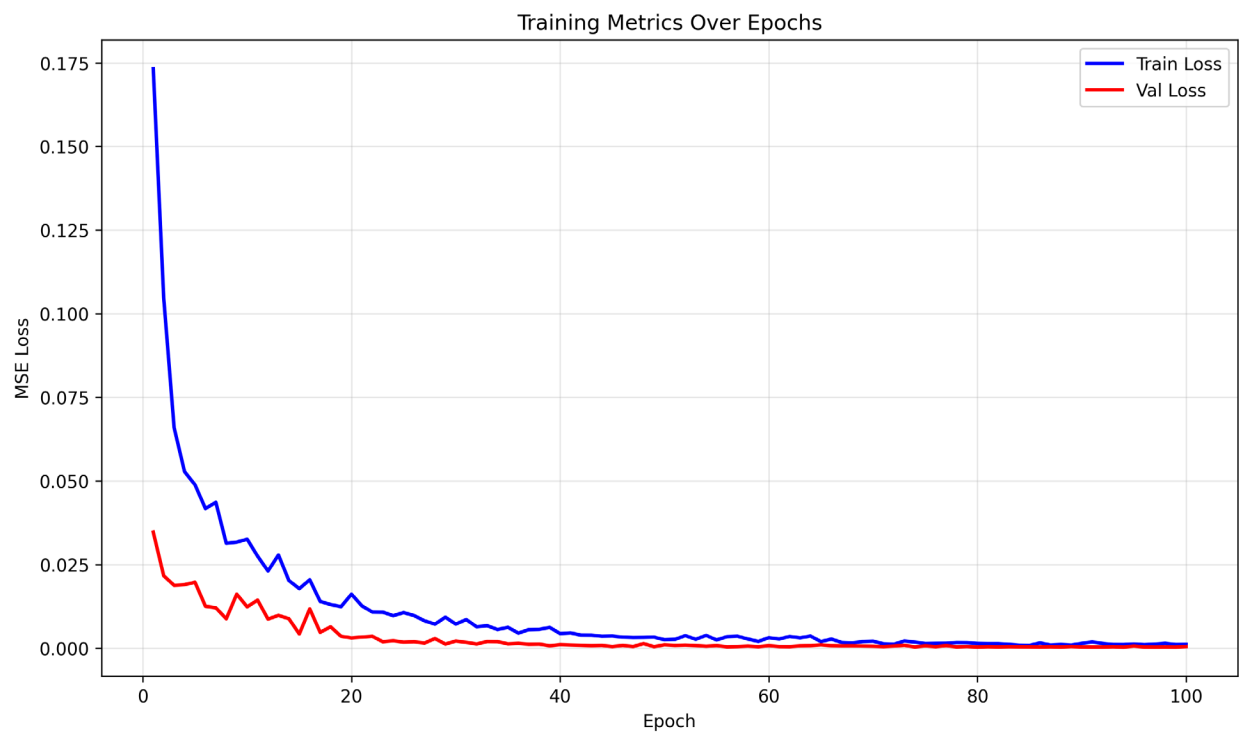


Figure 7. BiLSTM [50,50,50] horizon=1 training plot

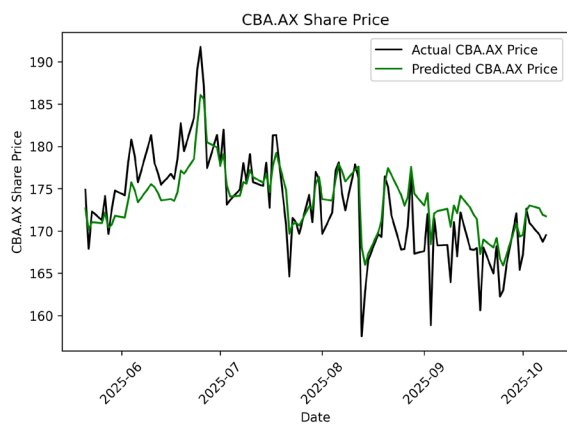


Figure 8. BiLSTM [50,50,50] horizon=1 prediction

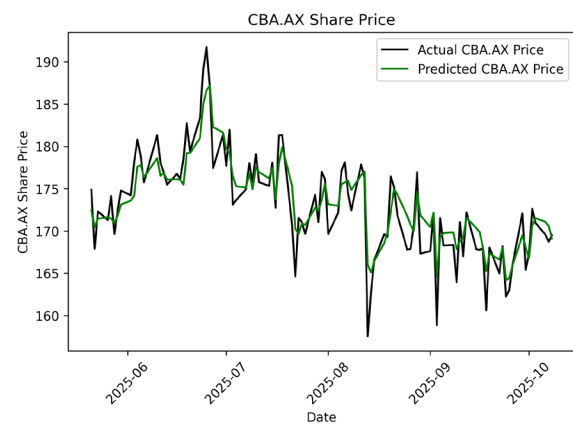


Figure 9. Ensemble (SARIMA:0.7 + BiLSTM:0.3) [50,50,50] horizon=1 (Great error/DA improvement)

4. GRU [50,50,50] horizon=5 (One of the best Directional Accuracy)

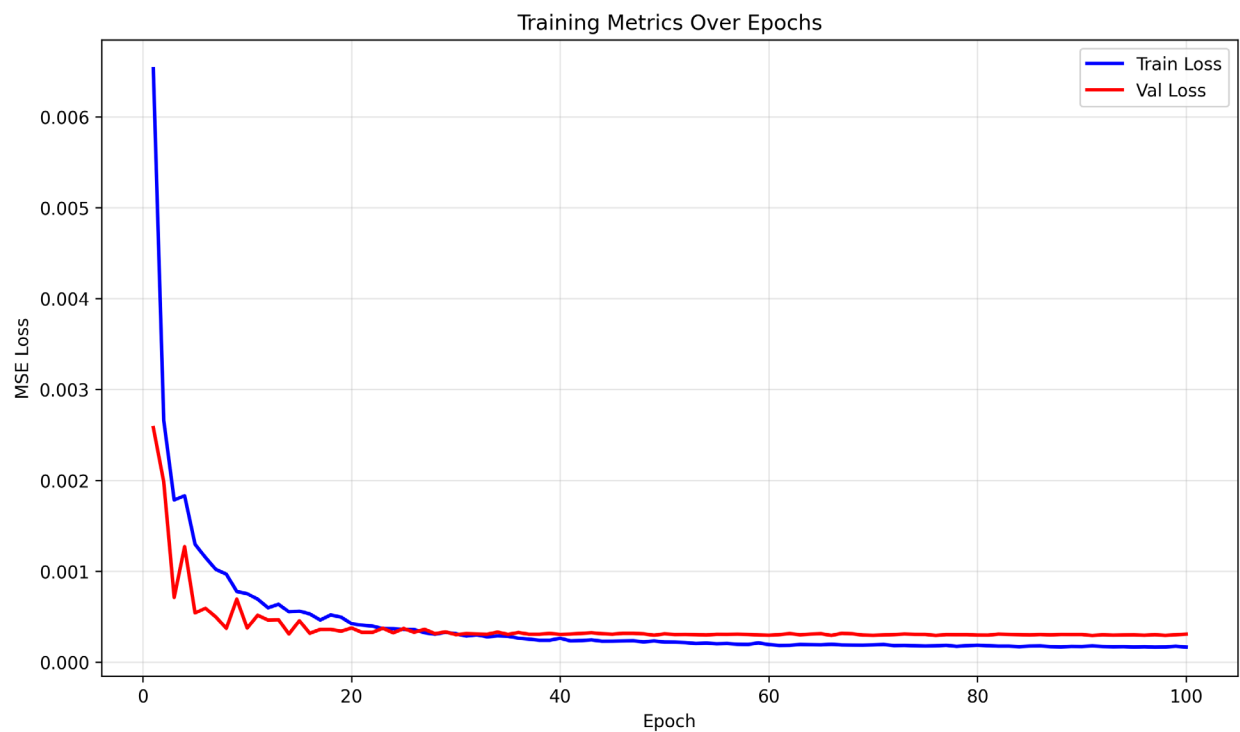


Figure 10. GRU [50,50,50] horizon=5 training plot

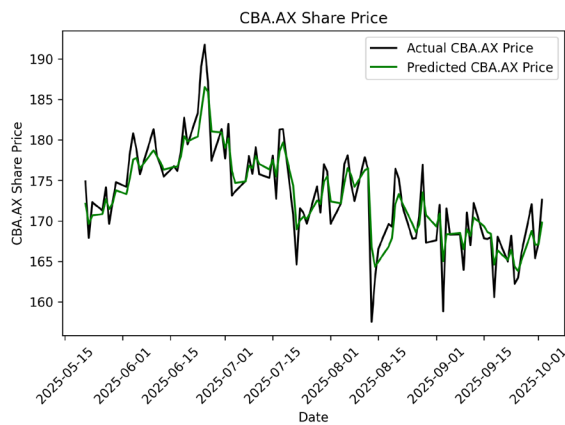


Figure 11. GRU [50,50,50] horizon=5 prediction

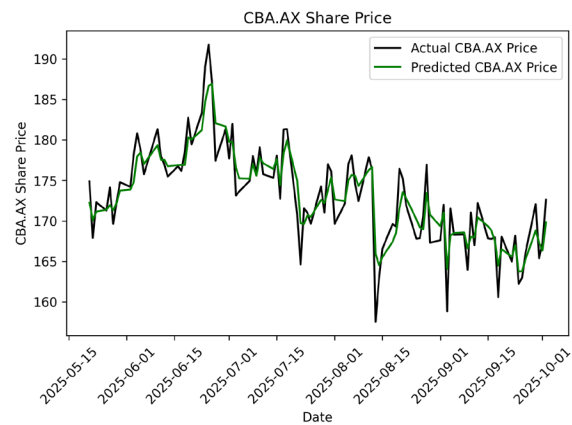


Figure 12. Ensemble (SARIMA:0.5 + GRU:0.5) [50,50,50] horizon=5 (single GRU wins)

5. LSTM [50,50,50] horizon=5 (Baseline)

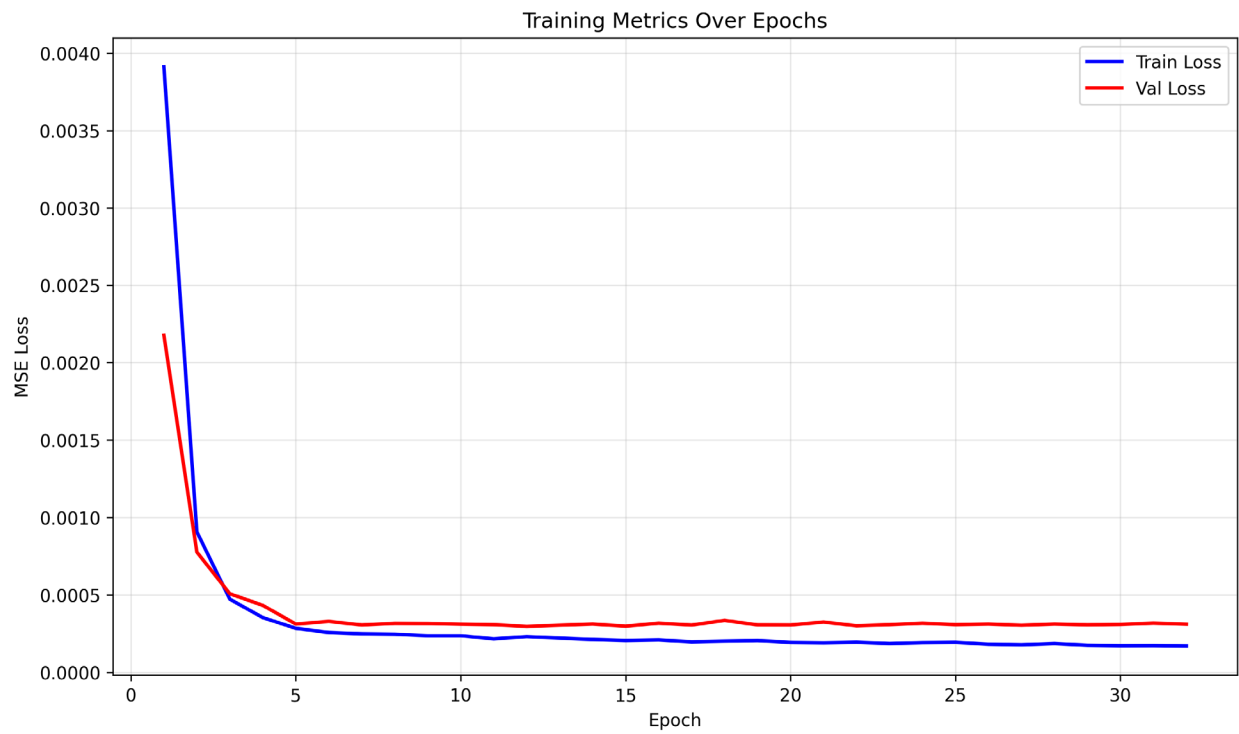


Figure 13. LSTM [50,50,50] horizon=5

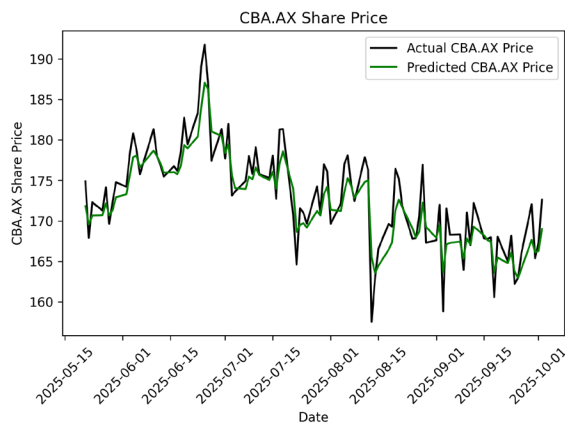


Figure 14. LSTM [50,50,50] horizon=5 prediction

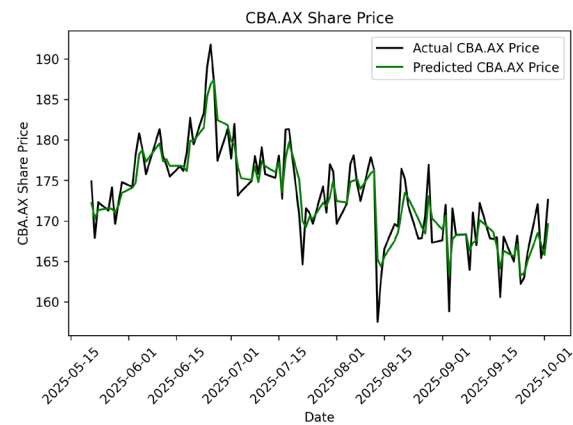


Figure 15 Ensemble (SARIMA:0.7 + LSTM:0.3) [50,50,50] horizon=5 (Highest ensemble DA)

Statistic summary and justification

Epochs = 100, batch sizes = 32, Optimizer = Adam, Learning rates=0.005, Lookback = 60

Model	Layers	Horizon	MAE	RMSE	DA	Epochs Trained
BiLSTM	[50, 50, 50]	1	2.768	3.439	0.747	32/100
BiLSTM	[64, 32]	1	2.840	3.532	0.737	36/100
BiLSTM	[50, 50, 50]	5	2.303	2.819	0.821	96/100
BiLSTM	[64, 32]	5	1.862	2.377	0.926	94/100
GRU	[50, 50, 50]	1	1.901	2.439	0.899	45/100
GRU	[64, 32]	1	2.190	2.849	0.828	100/100
GRU	[50, 50, 50]	5	1.773	2.316	0.937	100/100
GRU	[64, 32]	5	1.825	2.372	0.926	70/100
LSTM	[50, 50, 50]	1	2.101	2.683	0.869	24/100
LSTM	[64, 32]	1	2.057	2.555	0.869	57/100
LSTM	[50, 50, 50]	5	1.914	2.431	0.926	32/100
LSTM	[64, 32]	5	1.822	2.385	0.916	86/100
RNN	[50, 50, 50]	1	1.728	2.276	0.889	100/100
RNN	[64, 32]	1	1.743	2.303	0.869	100/100
RNN	[50, 50, 50]	5	1.742	2.281	0.937	96/100
RNN	[64, 32]	5	1.859	2.445	0.895	100/100
Ensemble (SARIMA-0.3 + BiLSTM-0.7)	[50, 50, 50]	1	2.315	2.910	0.818	-
Ensemble (SARIMA-0.3 + BiLSTM-0.7)	[64, 32]	1	2.347	2.946	0.778	-
Ensemble (SARIMA-0.3 + BiLSTM-0.7)	[50, 50, 50]	5	2.052	2.562	0.905	-
Ensemble (SARIMA-0.3 + BiLSTM-0.7)	[64, 32]	5	1.848	2.339	0.916	-
Ensemble (SARIMA-0.5 + BiLSTM-0.5)	[50, 50, 50]	1	2.064	2.639	0.848	-
Ensemble (SARIMA-0.5 + BiLSTM-0.5)	[64, 32]	1	2.099	2.644	0.828	-
Ensemble (SARIMA-0.5 + BiLSTM-0.5)	[50, 50, 50]	5	1.853	2.339	0.905	-
Ensemble (SARIMA-0.5 + BiLSTM-0.5)	[64, 32]	5	1.853	2.339	0.905	-
Ensemble (SARIMA-0.7 + BiLSTM-0.3)	[50, 50, 50]	1	1.912	2.456	0.879	-
Ensemble (SARIMA-0.7 + BiLSTM-0.3)	[64, 32]	1	1.936	2.443	0.869	-
Ensemble (SARIMA-0.7 + BiLSTM-0.3)	[50, 50, 50]	5	1.880	2.396	0.895	-
Ensemble (SARIMA-0.7 + BiLSTM-0.3)	[64, 32]	5	1.879	2.360	0.916	-
Ensemble (SARIMA-0.3 + GRU-0.7)	[50, 50, 50]	1	1.830	2.346	0.919	-
Ensemble (SARIMA-0.3 + GRU-0.7)	[64, 32]	1	1.969	2.572	0.859	-
Ensemble (SARIMA-0.3 + GRU-0.7)	[50, 50, 50]	5	1.790	2.305	0.926	-
Ensemble (SARIMA-0.3 + GRU-0.7)	[64, 32]	5	1.825	2.335	0.916	-
Ensemble (SARIMA-0.5 + GRU-0.5)	[50, 50, 50]	1	1.803	2.321	0.909	-
Ensemble (SARIMA-0.5 + GRU-0.5)	[64, 32]	1	1.900	2.448	0.859	-
Ensemble (SARIMA-0.5 + GRU-0.5)	[50, 50, 50]	5	1.823	2.319	0.916	-

Ensemble (SARIMA-0.5 + GRU-0.5)	[64, 32]	5	1.842	2.337	0.905	-
Ensemble (SARIMA-0.7 + GRU-0.3)	[50, 50, 50]	1	1.815	2.326	0.899	-
Ensemble (SARIMA-0.7 + GRU-0.3)	[64, 32]	1	1.875	2.379	0.879	-
Ensemble (SARIMA-0.7 + GRU-0.3)	[50, 50, 50]	5	1.864	2.351	0.916	-
Ensemble (SARIMA-0.7 + GRU-0.3)	[64, 32]	5	1.873	2.359	0.905	-
Ensemble (SARIMA-0.3 + LSTM-0.7)	[50, 50, 50]	1	1.917	2.467	0.879	-
Ensemble (SARIMA-0.3 + LSTM-0.7)	[64, 32]	1	1.888	2.402	0.909	-
Ensemble (SARIMA-0.3 + LSTM-0.7)	[50, 50, 50]	5	1.848	2.360	0.926	-
Ensemble (SARIMA-0.3 + LSTM-0.7)	[64, 32]	5	1.815	2.350	0.916	-
Ensemble (SARIMA-0.5 + LSTM-0.5)	[50, 50, 50]	1	1.842	2.379	0.929	-
Ensemble (SARIMA-0.5 + LSTM-0.5)	[64, 32]	1	1.843	2.347	0.909	-
Ensemble (SARIMA-0.5 + LSTM-0.5)	[50, 50, 50]	5	1.843	2.346	0.926	-
Ensemble (SARIMA-0.5 + LSTM-0.5)	[64, 32]	5	1.839	2.350	0.926	-
Ensemble (SARIMA-0.7 + LSTM-0.3)	[50, 50, 50]	1	1.831	2.342	0.899	-
Ensemble (SARIMA-0.7 + LSTM-0.3)	[64, 32]	1	1.827	2.333	0.899	-
Ensemble (SARIMA-0.7 + LSTM-0.3)	[50, 50, 50]	5	1.862	2.359	0.937	-
Ensemble (SARIMA-0.7 + LSTM-0.3)	[64, 32]	5	1.874	2.368	0.926	-
Ensemble (SARIMA-0.3 + RNN-0.7)	[50, 50, 50]	1	1.750	2.271	0.899	-
Ensemble (SARIMA-0.3 + RNN-0.7)	[64, 32]	1	1.745	2.277	0.889	-
Ensemble (SARIMA-0.3 + RNN-0.7)	[50, 50, 50]	5	1.779	2.297	0.926	-
Ensemble (SARIMA-0.3 + RNN-0.7)	[64, 32]	5	1.840	2.378	0.884	-
Ensemble (SARIMA-0.5 + RNN-0.5)	[50, 50, 50]	1	1.784	2.287	0.889	-
Ensemble (SARIMA-0.5 + RNN-0.5)	[64, 32]	1	1.774	2.285	0.879	-
Ensemble (SARIMA-0.5 + RNN-0.5)	[50, 50, 50]	5	1.820	2.322	0.926	-
Ensemble (SARIMA-0.5 + RNN-0.5)	[64, 32]	5	1.850	2.363	0.884	-
Ensemble (SARIMA-0.7 + RNN-0.3)	[50, 50, 50]	1	1.830	2.317	0.899	-
Ensemble (SARIMA-0.7 + RNN-0.3)	[64, 32]	1	1.821	2.313	0.899	-
Ensemble (SARIMA-0.7 + RNN-0.3)	[50, 50, 50]	5	1.869	2.357	0.916	-
Ensemble (SARIMA-0.7 + RNN-0.3)	[64, 32]	5	1.877	2.372	0.895	-

Key Findings:

Performance Rankings:

- **Best overall (lowest MAE & RMSE):** RNN [50,50,50], horizon=1 - MAE 1.728, RMSE 2.276, DA 88.9%.
- **Best multi-step (horizon=5):** RNN [50,50,50] - MAE 1.742, RMSE 2.281, DA 93.7%.
- **Best directional accuracy (overall):** 93.7% (tie) - GRU [50,50,50], h=5 and RNN [50,50,50], h=5 and SARIMA-0.7 + LSTM-0.3 - [50,50,50] h=5.
- **Best ensemble (lowest MAE):** SARIMA-0.3 + RNN-0.7 - [64,32] h=1: MAE 1.745, RMSE 2.277, DA 0.889.
- **Best ensemble (highest DA):** SARIMA-0.7 + LSTM-0.3 - [50,50,50] h=5: DA 0.937 (MAE 1.862, RMSE 2.359).
 - For **h=1**, the highest ensemble DA is 0.929 with SARIMA-0.5 + LSTM-0.5 ([50,50,50], MAE 1.842, RMSE 2.379).

Architecture Insights:

- **Deeper stacks [50,50,50]** generally outperform shallower [64,32] configurations for single-step predictions
- **Shallower stacks [64,32]** show competitive performance for multi-step horizons (e.g., BiLSTM [64,32] horizon=5: MAE 1.862, DA 92.6%)
- Simple **RNN architectures** achieve competitive or better results than complex LSTM/BiLSTM variants
- **LSTM** tends to underperform and stop early (24-57 epochs) and underperform RNN/GRU on this small dataset.

Training Dynamics:

- **Early stopping** is highly effective for single-step predictions (BiLSTM: 32-36, LSTM: 24-57, GRU: 45 epochs)
- **Multi-step forecasting (h=5)** requires longer training (70-100 epochs) to converge
- **RNN models** consistently train to completion (96-100 epochs), showing stable optimization
- **Ensemble epochs not tracked** due to cached submodels which already analyzed earlier.

Forecasting Horizon Impact:

- **Single-step (h=1)** predictions favor simpler architectures (RNN, Ensemble)
- **Multi-step (h=5)** benefits from GRU/RNN gating/recurrence.
- **Directional accuracy** remains high (>90%) across horizons for well-trained models
- Performance **does not always degrade** with longer horizons when using appropriate architectures (e.g., GRU [50,50,50])

Ensemble model vs single model comparison:

Experiments (lookback=60) show the best single model is RNN [50,50,50] (h=1: MAE 1.728; h=5: MAE 1.742), while ensembles mainly boost weaker bases (e.g., LSTM/BiLSTM) and can deliver top directional accuracy (e.g., SARIMA-0.7 + LSTM-0.3, h=5: DA 0.937).

Base	Horizon	Best Single (MAE, DA)	Best Ensemble (weights, MAE, DA)	Δ MAE ($\downarrow$ better)	Δ DA ($\uparrow$ better)	Note
BiLSTM [50,50,50]	1	2.768, 0.747	(0.7/0.3) $\rightarrow$ 1.912, 0.879	-0.856 (-30.9%)	+0.132	Big gain on both MAE & DA
BiLSTM [64,32]	5	1.862, 0.926	(0.3/0.7) $\rightarrow$ 1.848, 0.916	-0.014 (-0.8%)	-0.010	Tiny MAE gain, slight DA drop
GRU [50,50,50]	1	1.901, 0.899	(0.5/0.5) $\rightarrow$ 1.803, 0.909	-0.098 (-5.2%)	+0.010	Clear improvement
GRU [50,50,50]	5	1.773, 0.937	(0.3/0.7) $\rightarrow$ 1.790, 0.926	+0.017 (+1.0%)	-0.011	Single GRU wins
LSTM [64,32]	1	2.057, 0.869	(0.7/0.3) $\rightarrow$ 1.827, 0.899	-0.230 (-11.2%)	+0.030	Big improvement, best DA with 0.5/0.5 (0.929)
LSTM [64,32]	5	1.822, 0.916	(0.5/0.5) $\rightarrow$ 1.839, 0.926	+0.017 (+0.9%)	+0.010	Slight MAE worse, DA better
RNN [50,50,50]	1	1.728, 0.889	(0.3/0.7) $\rightarrow$ 1.745, 0.889 (or 1.750, 0.899)	+0.017 (+1.0%)	0 (or +0.010)	Single RNN wins MAE, ensemble can lift DA slightly
RNN [50,50,50]	5	1.742, 0.937	(0.3/0.7) $\rightarrow$ 1.779, 0.926	+0.037 (+2.1%)	-0.011	Single RNN wins

7. Conclusion

I refactored the codebase into a validated, single-source-of-truth pipeline (DataBundle + converters), added SARIMA and a weighted ensemble, and introduced train/val/test with early stopping.